

Foodly: Core System Design & Architecture Issues Report

1. System Design Overview

The Foodly project is designed as a modern, AI-first Monolith MVP (Layer 1 moving into Layer 2), structured around the following stack:

- **API Framework: FastAPI**, chosen for its high-performance asynchronous capabilities and automatic API documentation.

Database: PostgreSQL, extended with two powerful plugins:

- **PostGIS:** Handles geospatial queries (e.g., `ST_DistanceSphere` used in the `/nearby` endpoint) to find meals based on user coordinates.
- **pgvector:** Enables semantic vector similarity search. Text (like meal descriptions) is converted into 1536-dimensional embeddings and stored in the database, allowing users to search by "meaning" rather than exact text matches.

AI & Agents Layer: Utilizes **LangChain** and **LangGraph** to build autonomous features.

- *LLM Providers:* Google Gemini (for value insights and embeddings) and Groq (for fast inference in data extraction).
- *Agents:* A LangGraph-based `price_scraper` orchestrates web searches (via Tavily) to verify meal prices autonomously.
- **Data Access (ORM): SQLAlchemy**, which translates Python objects into SQL queries and manages the connection pool.

2. Core System Design & Architecture Issues

Despite the solid foundation, there are several significant architectural flaws in the current implementation that violate production standards and the guidelines set in the execution plan:

A. Severe Service Layer Violations (Tight Coupling)

- **The Issue:** The execution plan strictly dictates a `Route -> Service -> Repository` architecture. However, in `api/endpoints/meals.py`, raw database queries (`db.query(...)`) and SQLAlchemy model filtering are written directly inside the FastAPI route handlers.
- **The Impact:** This tightly couples the HTTP layer to the database schema. It makes unit testing nearly impossible without mocking the entire database, and business logic (like computing price history) cannot

be reused elsewhere in the app.

B. Blocking I/O in an Asynchronous Framework

- **The Issue:** FastAPI is built to be asynchronous. However, the AI insight generator (`generate_value_insight` in `agents.py`) uses a synchronous `llm.invoke()` call.
- **The Band-Aid:** `meals.py` currently attempts to fix this by throwing the call into a `ThreadPoolExecutor` (`_ai_executor.submit`).
- **The Impact:** While this prevents the main event loop from deadlocking, relying on thread pools for long-running network calls (like an AI API) in a high-traffic app leads to thread exhaustion. It is not production-safe. The AI calls must be refactored to use native async methods (e.g., `ainvoke()`).

C. Flawed LangGraph Logic & Missing "Write" Operations

- **The Issue:** In `backend/ai/agents/price_scraper.py`, the `should_continue` edge is broken. If it fails to find a price, it returns "continue", which just loops back and runs the *exact same search query* again, yielding the exact same failure. There is no LLM "reasoning" step to refine or expand the search query.
- **The Impact:** The LangGraph implementation is effectively a static script disguised as an agent. Furthermore, when it *does* find a price, it merely prints it to the console. It does not write the result to the `pending_verifications` database table as required.

D. In-Memory Caching vs. Distributed Caching

- **The Issue:** In `meals.py`, AI insights are cached using Python's native `@lru_cache`.
- **The Impact:** When deployed to production, FastAPI will likely run across multiple worker processes (e.g., 4 Gunicorn workers). Because `@lru_cache` is bound to the individual process memory, Worker A will not know what Worker B has cached. This results in terrible cache hit rates, redundant calls to the Gemini API, and wasted API costs. A distributed cache like **Redis** is required here.

E. Brittleness in Semantic Search

- **The Issue:** The semantic search in `meals.py` takes the user's exact string, embeds it, and checks for proximity. If a user searches "Desi Food", the vector might completely miss specific meals like "Biryani" or "Karahi" because the literal mathematical vector of the words is too narrow.
- **The Impact:** Low search recall. The architecture is missing the **Query Expansion** step (Day 11 of the plan) where an LLM first breaks down "Desi Food" into an array of specific localized terms before embedding them.

3. Recommended Remediation Steps

1. **Repository Pattern:** Move all `db.query()` logic out of `meals.py` and into a `MealRepository` class.
2. **Async Refactor:** Refactor `agents.py` to use `await llm.ainvoke()` natively to avoid blocking the event loop.
3. **LangGraph Fixes:** Fix the LangGraph retry state to actually append context, rewrite the query on failure, and persist successful hits to the database.
4. **Caching:** Replace `@lru_cache` with a Redis integration.
5. **Query Expansion:** Introduce an LLM-based query expansion chain before performing semantic searches.